

Meridian UAV Services — Predictive Maintenance

Two numbers in the wrong unit

Analyst case study · Reporting date 2026-09-30

Stack: T-SQL (SQL Server) · Python (scikit-learn) · Streamlit · Excel

Data disclosure. Meridian UAV Services is a fictional UAV operator. All data is synthetic and generated by the scripts in `sql/`. Degradation behaviour is modelled on publicly described characteristics of rotating machinery. **No confidential data is used and no claim is made about any real aircraft, operator, manufacturer or production system.**

The business question

Meridian operates 28 unmanned aircraft from four bases. Every flight-critical component carries a service interval, and the maintenance system reports the fleet at **99.73% compliance**.

The operations director does not believe it. Unscheduled removals are running at **36%** of all maintenance events, the hangar is chronically behind, and the aircraft that fail are not the aircraft the interval report flags. The question put to analysis was blunt:

If we are 99.73% compliant, why do we keep having failures?

There are two findings. They are about different things, and they have the same shape.

Finding 1 — The fleet is compliant in a unit that does not measure wear

An hour is not a unit of damage.

An hour of mountain survey at 22 kg and an hour of coastal survey at 12 kg do not damage a rotor bearing by the same amount. An hour of circuit training produces ten landings; an hour of cargo relay produces one. The interval is enforced in **flight hours** because that is what the hour meter produces — not because that is how the part wears.

Ref_StressModel converts each sortie into **stress-weighted duty cycles**:

```
stress hours = flight hours × profile multiplier
               + landings    × cycle penalty
               + flight hours × (payload – 12 kg) × payload penalty
```

Measure the same components, the same intervals, on the same day, in that unit instead:

	Flight hours	Stress hours
Fleet compliance	99.73%	68.41%
Flight-critical components past due	1	90

114 components are past their interval in the unit their wear accrues in, 89 of them flight-critical, and every one of them reads as compliant on every report the operator runs.

The most extreme is **MU-008** at Highland: its rotor assembly is at **93.16%** of its flight-hour interval and **205.46%** of its stress interval. The maintenance system has it down for replacement in about three weeks. It is at twice its life.

This is not a Highland problem. It is a work-mix problem.

Base	Airframes	Stress per flight hour	Hour compliance	Stress compliance	Gap	Unscheduled
Highland	7	2.1532	100.00%	62.64%	37.36	51.01%
Port	7	2.0040	98.90%	57.14%	41.76	40.14%
Academy	4	1.8594	100.00%	57.69%	42.31	41.67%
Coastal	10	1.1741	100.00%	84.62%	15.38	17.86%

Read the first numeric column and the rest of the table follows from it. Highland flies mountain and cargo work and accrues **2.15 stress hours per logged flight hour**; Coastal flies survey and accrues **1.17**. The hour meter cannot tell them apart, and the interval treats them identically.

Coastal is the control. Its aircraft run at 84.62% stress compliance and 17.86% unscheduled — a normally-run fleet. The 150-hour interval genuinely suits the work it flies. The policy is not broken everywhere; it is broken for the work it was never calibrated against.

The Academy is the one to look at twice

The training base flies the **lightest payloads and the fewest hours in the fleet**. By every measure the maintenance system records, it is the easiest life an airframe can have. It has the **worst stress**

compliance of the four bases at 57.69%.

Circuit training produces six to fifteen landings per sortie against one or two for cargo relay, and the cycle penalty is what dominates its stress total. An operator managing by the hour meter would conclude the academy aircraft are barely used, and would be exactly wrong.

Finding 2 — The failure alarm is accurate and mostly useless

Meridian runs condition monitoring on rotor assemblies and motor controllers: an alarm fires when vibration reaches a fixed multiple of the component's own baseline. Against the programme's published targets it is a success.

Metric	Value	Target	
Prediction precision	81.56%	75.00	Green
Prediction recall	98.26%	80.00	Green
Actionable lead time	37.81%	70.00	Red

Of **283 failures the alarm correctly predicted, 176 arrived too late to act on.**

A rotor assembly takes 18 flight hours to obtain and fit. An alert with six hours of warning is a correct prediction and a grounded aircraft — and precision and recall cannot tell those two alerts apart, because both are true positives. **A prediction is worth its lead time, not its accuracy.**

The cause is one threshold across two different physics

At the deployed 3.40× threshold	Precision	Actionable	Median warning	Part lead time
MOTOR-ESC	97.17%	8.74%	6.75 h	12 h
ROTOR-ASSY	74.69%	54.44%	18.93 h	18 h

A rotor bearing's vibration rises gradually from about half its life. A motor controller's stays flat and then turns up sharply past 85%. One threshold cannot suit both, and at the setting that makes **rotor** precision acceptable, the **motor-controller** alarm becomes almost perfectly accurate and almost entirely useless: 94 of its 103 correct predictions arrive too late to order the part.

The fix is a number, and it is free

Choosing each threshold on **actionable lead time subject to a 75% precision floor** — rather than on precision, or on F1:

Component	Threshold	Precision	Actionable	Failures caught in time
MOTOR-ESC	3.400 → 2.050	97.17% → 75.18%	8.74% → 93.20%	9 → 96 (+87)
ROTOR-ASSY	3.400 → 3.450	74.69% → 75.63%	54.44% → 51.11%	98 → 92 (-6)
Fleet	one → per component	81.56% → 75.47%	37.81% → 66.43%	107 → 188

81 more failures caught early enough to prevent, for six points of fleet precision. The alarm hardware does not change. The telemetry does not change. One number per component changes.

Is the warning actually there, or is the data just late?

The obvious objection is that the information does not exist early enough, and no threshold can create it. That was tested rather than assumed, by fitting three models over the same test components. Their features enter at the resolution the sensors actually have (two or three decimals), not the generator's full precision, so no model can learn from digits a real instrument would never report:

Inputs	AUC	Precision	Actionable	Median warning
Telemetry only — vibration, temperature, current	0.9781	75.63%	92.22%	26.85 h
Exposure only — accrued stress, stress ratio	0.6798	<i>never reaches 75%</i>	—	—
Both	0.9986	85.71%	100%	43.50 h
<i>Deployed fixed 3.40× threshold</i>	—	77.39%	38.20%	13.18 h

None of these figures is one lucky draw. The model holds out a random slice of its training readings for early stopping, so `python/seed_sensitivity.py` retrains every row under ten seeds. Telemetry clears the 75% precision floor under all ten, at 88.89%–97.78% actionable (median 92.78%); exposure clears it under none (AUC 0.677–0.685); the published seed reproduces the table exactly. The comparison, not the second decimal, is the finding.

Two things follow.

The warning is in the data. Telemetry alone, read by a model instead of a fixed threshold, recovers 92% actionable lead time at essentially the same precision. The deployed rule discards it.

The stress model is not a substitute for condition monitoring. Exposure features alone — Finding 1's variables — cannot reach the precision floor at any threshold. The two findings are about different things and the operator needs both. That is worth stating precisely because the tempting conclusion from Finding 1 is that the stress model makes the telemetry redundant. It does not.

What to do on Monday

The action queue ranks every fitted component: flight-critical first, already-overdue before merely projected, then furthest past its limit — with a stable tie-break, because a queue whose rows move between refreshes cannot be worked through.

194 outstanding jobs. 682.5 hangar hours. 5.7 weeks at the current 120 hours per week.

Action	Jobs	What it needs
GROUND_AND_REPLACE	90	A bay and a technician. Flight-critical and already past its stress interval.
SCHEDULE_THIS_MONTH	54	A slot inside 40 flight hours.
ORDER_PART_NOW	25	A purchase order, not a bay. Projected to reach its limit inside the part lead time.
REPLACE_AT_NEXT_SLOT	25	Past due, not flight-critical.

The action follows the state, not the rank. `ORDER_PART_NOW` costs zero hangar hours because it is procurement — a different person, a different day, and no bay time at all. Lumping it into "overdue" loses the only distinction that decides who acts.

26 jobs fit in the first week. That is the honest schedule answer, and changing the capacity cell in the workbook re-cuts it: at 240 hours a week it becomes 53 jobs and 2.8 weeks.

Recommendations

- **Re-express the service intervals in stress hours.** `Ref_ServiceInterval` already carries both columns. The maintenance system continues to enforce hours; the planning layer schedules on stress. No aircraft change, no new instrumentation.
 - **Retune the motor-controller alarm to 2.05x.** It is strictly better on the axis that matters and still clears the precision floor. This is the single highest-return action in the analysis: 87 additional failures caught in time, for nothing.
 - **Do not retune the rotor alarm — stock the part.** At a 75% precision floor the rotor cannot get past 51% actionable, because its median warning (18.93 h) sits almost exactly on its 18-hour lead time. Tuning cannot fix that. Holding rotor assemblies in local stores can, and it is a supply decision rather than an analytics one.
 - **Manage Highland, Port and Academy on a shorter interval than Coastal.** The work differs by 2.15 : 1.17 in damage per logged hour. A single fleet-wide interval will keep producing this.
 - **Stop reporting fleet-average compliance.** It is 99.73% and it describes none of the four bases.
-

How the analysis was built

The dataset

31 airframes, 10,501 sorties, 1,549 component installs, 83,877 condition-monitoring readings and 1,185 maintenance events across 33 months — all synthetic, generated by `sql/02`.

The findings are not planted. Each component is given a life in stress hours drawn from a Weibull distribution with the shape its type exhibits, and then accrues stress from the sorties its airframe happens to fly. It is removed by whichever of two clocks finishes first: the **maintenance clock** (flight hours reaching the interval) or the **physics clock** (stress reaching its drawn life). Highland's aircraft fail more because of the work they fly, not because anything says they should.

The generator is **deterministic**: every draw is a SHA2-256 hash of a stable business key, so the dataset regenerates byte for byte and every figure above stays checkable.

The modelling decision everything rests on

Accrued component life is a dated fact computed in a window, never an attribute stored on the row.

"Hours on this component" is not a property of the component. It is a question with a date in it, and the answer changes every time the aircraft flies. Storing it would mean maintaining it, and it would go quietly stale.

The same decision governs the population: **fleet membership is a date question, not a status flag.** Three aircraft retired inside the reporting window, and a compliance figure for June 2025 must include them. Filtering on `AirframeStatus = 'Active'` — a current-state attribute — computed the entire eighteen-month trend over the 28 aircraft flying today. That defect shipped, passed the test suite, and was caught by adversarial review.

Data quality

Twelve behavioural checks. Nothing is flagged in the data; every check finds its defects by behaviour, so it would work on a real extract where nobody labelled anything. **Seven found nothing and are reported anyway** — a summary listing only what it found cannot distinguish *we looked and it was clean* from *we never looked*.

73 findings, 12 of which touch a figure in today's answer.

The gate does not count defective rows. It compares each flight-critical component's **uncertainty band** — the stress it cannot stand behind — against its **margin** to the threshold, and asks whether correcting the data could move it across the line. **It passes at 0.000% against a 3.000% tolerance.** The first version of that gate measured "touched by flagged data" and failed at 22.50%, a number with no decision attached.

Testing

35 acceptance cases, all passing. Every case asserts a **value** where one is knowable, not a property — `compliance is between 0 and 100` holds under the very bug it was written to catch.

Six cases break something deliberately and assert the detection fires. The suite also asserts its own expected case count, which is how it caught two cases silently vanishing: their results were written inside a fixture transaction, and the rollback discarded them along with the fixture. Both had executed. Both had passed. Without the count guard the run would have reported "24 passed, 2 failed" with nothing saying two cases had disappeared.

Full detail in `docs/data_validation_report.md` and `docs/requirements_traceability_matrix.md`.

The deliverables

`excel/Meridian_Fleet_Control.xlsx` — **a control, not a report.** Nothing on the Dashboard is a pasted number: both sides of the headline comparison are computed in Excel with `COUNTIFS` over the component extract. A comparison where one side is copied from the tool that produced the other proves nothing about either. All 16 on-sheet checks reconcile against SQL, and the validator **reopens the saved file, recalculates it, and asserts the controls change the answer** — doubling the capacity cell moves the week from 26 jobs to 53; setting the threshold to 2.05 moves motor-controller actionable lead time from 8.74% to 93.2%, matching SQL exactly.

`streamlit/app.py` — **five screens, each ending in a decision.** It contains no analytical logic: every definition arrives already computed, and the two interactive controls are built so they cannot duplicate it — the capacity slider re-cuts a queue SQL already ranked, and the threshold slider reads a sweep SQL already evaluated.

`python/degradation_model.py` implements the alarm rule a second time, in pandas, and requires it to agree with T-SQL exactly on population, true positives, false positives, false negatives and actionable count. That reconciliation earned its keep: it reported 2 false negatives where T-SQL reported 3. T-SQL was right — pandas had built its population from the *readings*, so a monitored component that produced **no telemetry at all** did not exist in its evaluation. A component nobody can see cannot be alerted on. **Invisibility is a miss**, and excluding it flatters recall.

What this analysis does not claim

The stress multipliers are the most contestable input here, and whether mountain work really costs 2.4× coastal is exactly the argument a maintenance engineer should be having. The numbers live in a table with their reasoning attached so that argument can be had against the table rather than against a `CASE` expression nobody can see.

The stress model has no component dimension. A landing costs a rotor assembly what it costs the landing gear, which is physically wrong. The comparison this project makes — two airframes flying different work accrue different damage per logged hour — does not depend on the missing dimension, but differences *between component types* come from their Weibull shapes, not from any component-specific response to landings.

A model trained on synthetic degradation learns the generator. The AUC figures are not a claim about real rotor bearings. What transfers is the method: evaluate on actionable lead time, split by component and by time, and establish whether the signal is in the telemetry or in the exposure.

Reconciliation proves agreement, not correctness. An earlier project in this portfolio shipped an Excel workbook and a SQL view that agreed exactly and were both wrong, because both came from the same flawed written definition.

The shape both findings share

Finding 1: the fleet is measured in flight hours, and wear does not accrue in flight hours.

Finding 2: the alarm is measured in precision and recall, and usefulness does not accrue in precision and recall.

Both are a correct number in the wrong unit, and in both cases the wrong unit was chosen because it was the one the instrument happened to produce — the hour meter, and the confusion matrix. Neither is a data problem, and neither is fixed by collecting more of anything.

The fleet already has the telemetry it needs. It already records landings and payload. The two changes that matter are a column and a threshold.